NATIONAL RESEARCH UNIVERSITY OF INFORMATION TECHNOLOGY, MECHANICS AND OPTICS

FACULTY OF CONTROL SYSTEMS AND ROBOTICS

Smooth Trajectory Planning for ABB IRB 140 Robot via State-Space Interpolation and Simulation in CoppeliaSim: A Comparative Study of Optimization Algorithms

ATP Project

COMPLETED BY:	**Valverde Ramírez, Edgar David**
ISU:	**520193**
GROUP:	**R4136c**
DISCIPLINE:	**Simulation of Robotic Systems**
INSTRUCTOR:	**PhD. Ivan Borisov**
ASSISTANT:	**Rakshin, Egor**

1. **PROJECT OVERVIEW:**
   This project implements a direct trajectory planning approach for the ABB IRB 140 industrial manipulator, transitioning from task space to joint space, the system utilizes state-space modeling where the control signal is the n-th derivative of the joint position.

   The primary objective is to generate smooth motions that minimize the final positioning error while strictly adhering to kinematic constraints (position and velocity limits). This implementation ignores dynamic constraints (torque).
   A key feature of this project is the **comparative implementation of two optimization strategies**:

   1. **Gradient-based:** Using scipy.optimize.minimize (SLSQP).
   2. **Evolutionary:** Using Genetic Algorithms with the DEAP library.

   Validation will be performed via simulation in CoppeliaSim, controlled externally via Python.

2. **OBJECTIVES:**
   1. **Kinematic Modeling:** Implement the Forward Kinematics (FK) of the ABB IRB 140 in Python using Denavit-Hartenberg (DH) parameters.
   2. **Interpolation Algorithm:** Develop a state-space interpolator utilizing Cauchy's formula for repeated integration to ensure mathematical continuity (C^2 or C^3).
   3. **Optimization Strategies:**
      - Implement **SLSQP** (Sequential Least Squares Programming) to find local minima efficiently.
      - Implement a **Genetic Algorithm (GA)** using DEAP to explore the search space and avoid local minima.
   4. **Comparative Analysis:** Evaluate both algorithms based on convergence time, solution quality, and constraint satisfaction.
   5. **Simulation:** Visualize the resulting smooth trajectories in CoppeliaSim using the Remote API (ZMQ).

3. **METHODOLOGY (Key Components)**
   Based on the "Integrated Approach to Robotic Joint Interpolation," the development is divided into:

   A. State-Space Modeling
   Unlike traditional interpolation (IK → Polynomials), a discrete system is defined:

$$X_{(i+1)} = \mathbb{A}X_{(i)} + \mathbb{B}U_{(i)}$$

   - **State (X):** Contains joint positions and their derivatives up to order $n-1$.

- **Control (U):** The n-th derivative of position (e.g., Jerk or Jounce).
- **Prediction Matrices ($\mathcal{M}, \aleph$):** Stacked system matrices used to predict the future state over a horizon k.

B. Kinematic Optimization

A cost function L is defined to minimize the deviation between the desired pose and the FK result at the end of the interval:

$$L = \| Y_{desired} - FK(X_{(k)}) \|^2 + \lambda \sum U^2$$

(A regularization term λ is added to penalize excessive control effort).

Constraints:

- $q_{min} \leq q(t) \leq q_{max}$ (Joint limits).
- $-\dot{q}_{max} \leq \dot{q}(t) \leq \dot{q}_{max}$ (Velocity limits).

Solvers:

1. **SciPy (SLSQP):** minimize(fun, u0, method='SLSQP', bounds=..., constraints=...). Best for fine-tuning and speed.
2. **DEAP (Genetic Algorithm):** Defined with individual chromosomes representing the control vector U.
    - *Selection:* Tournament.
    - *Crossover:* Two-point or blend crossover.
    - *Mutation:* Gaussian mutation.

C. Integration (Cauchy Formula)

Once the control signal U is optimized, the joint position trajectories $q(t)$ are reconstructed by integrating U, n times using Cauchy's formula, ensuring the trajectory is smooth and differentiable.

4. IMPLEMENTATION PLAN (1 Week)

Day	Phase	Key Activities
1	**Setup & Kinematics**	• Configure Python environment (NumPy, SciPy) and CoppeliaSim scene with the ABB IRB 140 model. • Define the IRB 140 DH table and code the Forward Kinematics (FK) function.
2	**Mathematical Framework**	• Implement the state-space matrices (A, B) and prediction matrices ($\mathcal{M}, \aleph$). • Program the Cauchy formula to reconstruct the trajectory from the control vector U.

3	**Optimization Algorithm**	• Define the loss function (Position/Orientation error). • Configure the scipy.optimize.minimize solver (SLSQP method) including the bounds for position and velocity.
4	**Numerical Testing**	• Run optimization tests without the simulator. • Verify via Matplotlib that the generated position and velocity graphs respect theoretical limits.
5	**Coppelia Integration**	• Connect Python to CoppeliaSim (ZeroMQ Remote API). • Send the optimized trajectory ($q_1 \dots q_6$ at each timestep) to the robot in the simulation.
6	**Refinement & Data**	• Tune penalty parameters in the optimization to smooth out motions. • Data collection: Error plots, computation time, and 3D path tracing.
7	**Final Delivery**	• Edit the demonstration video. • Write the final report and document the code

5. MILESTONES & DELIVERABLES

Milestone	Date (Day)	Description	Success Criteria
M1: FK & State Space	Day 2	Functional kinematics module and state prediction.	FK Error < 10^{-5}m compared to CoppeliaSim.
M2: Optimizer	Day 4	Algorithm converges to a valid solution without violating limits.	Trajectories respect V_{max} and reach the target point.
M3: Simulation	Day 6	Visual execution in CoppeliaSim.	Robot follows the trajectory smoothly without jerky movements.
M4: Submission	Day 7	Final Package (Code + Report + Video).	All materials uploaded and documented.

6. PERFORMANCE TARGETS

Metric	Baseline (Linear Interpolation)	Target (Paper Method)	Improvement
Final Position Error	N/A (IK is exact)	< 1.0 mm	High arrival precision
Smoothness (Jerk)	High (Discontinuous at nodes)	Continuous (C^2 or C^3)	Fluid motion, no vibrations
Velocity Compliance	Not guaranteed (if t is short)	100% Guaranteed	Operational safety (Hard constraints)
Computation Time	< 10$ ms	< 2 s (Offline)	Complex optimization but viable for planning